AI-BASED JOB RESUME SCREENING SYSTEM

MINI PROJECT REPORT

Submitted to the

SCHOOL OF ENGINEERING

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

in Partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Submitted by

ARJUN N

BALA SANKAR R

JOTHESHWAR R

VELS INSTITUTE OF SCIENCE, TECHNOLOGY AND ADVANCED STUDIES (VISTAS)

CHENNAI – 600 117

OCTOBER 2026

BONAFIDE CERTIFICATE

Certified that this project report “AI-BASED JOB RESUME SCREENING SYSTEM” is the bonafide work of ARJUN N, BALA SANKAR R and JOTHESHWAR R, carried out in the Department of Artificial Intelligence and Machine Learning, School of Engineering, Vels Institute of Science, Technology & Advanced Studies (VISTAS), Chennai.

The work presented in this report is submitted in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Artificial Intelligence and Machine Learning.

SIGNATURE OF THE HEAD OF THE DEPARTMENT

Name: _____
Designation: _____

SIGNATURE OF THE PROJECT GUIDE

Name: _____
Designation: _____

Submitted for the Project Work and Viva-Voce examination held on _____ at Vels Institute of Science, Technology and Advanced Studies (VISTAS), Chennai – 600 117.

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We express our sincere gratitude to Vels Institute of Science, Technology & Advanced Studies (VISTAS), Chennai, for providing the facilities and academic environment required to complete this mini project.

We thank the Head of the Department of Artificial Intelligence and Machine Learning for encouragement and guidance throughout the project. We are grateful to our project guide for valuable suggestions and support during the planning and preparation of this report.

We also thank the faculty members, laboratory staff, classmates, friends, and family members who supported us directly or indirectly. Their feedback helped us understand the practical challenges involved in using artificial intelligence for recruitment-related tasks.

Finally, we acknowledge the developers and maintainers of the open-source software and learning resources that make experimentation with Python, natural-language processing, and machine-learning techniques possible.

ARJUN N

BALA SANKAR R

JOTHESHWAR R

ABSTRACT

Recruiters may receive hundreds of resumes for a single job opening. Manually reviewing every application can require considerable time and may lead to inconsistent shortlisting. This project proposes an AI-Based Job Resume Screening System that helps organize and compare resumes against a job description using transparent, job-related criteria.

The prototype accepts resume text and a job description, cleans and tokenizes the text, extracts relevant skills and keywords, and calculates a match score. The score can combine skill coverage, relevant experience terms, education requirements, and semantic similarity. Applicants are then placed into a review queue so that a recruiter can examine the evidence behind each score.

A possible implementation uses Python, regular expressions, TF-IDF or sentence embeddings, and a lightweight web interface. The system is designed as a decision-support tool rather than an automatic rejection mechanism. It should not infer protected characteristics, personality, health, age, gender, caste, religion, or other sensitive traits from resumes.

The report describes the problem, literature overview, system design, requirements, methodology, implementation plan, sample code, testing strategy, limitations, and future enhancements. Because no real recruitment dataset or validated experiment is supplied with this report, any example scores are demonstrations only. Actual use requires representative evaluation, privacy safeguards, accessibility checks, and human review.

Keywords: Artificial Intelligence, Resume Screening, Natural Language Processing, Machine Learning, Skill Matching, Recruitment Automation, Fairness, Explainable AI.

TABLE OF CONTENT

CHAPTER / SECTION	TITLE	PAGE NO.
—	Bonafide Certificate	2
—	Acknowledgement	3
—	Abstract	4
—	Table of Contents	5
—	List of Figures and Abbreviations	6
1	Introduction	7–10
2	Literature Review	11–13
3	System Design	14–18
4	Implementation and Testing	19–25
5	Conclusion and Future Work	26–28
—	Appendix: Sample Code	29
—	References	30

Note: Page numbers refer to this 30-page report layout. If the report is edited or reformatted, update the contents page accordingly.

LIST OF FIGURES AND ABBREVIATIONS

List of Figures

• Figure 1. High-level architecture of the resume screening system

• Figure 2. Resume screening workflow

• Figure 3. Illustrative recruiter review dashboard

Abbreviations

Abbreviation	Full form
AI	Artificial Intelligence
ML	Machine Learning
NLP	Natural Language Processing
CV	Curriculum Vitae
TF-IDF	Term Frequency–Inverse Document Frequency
API	Application Programming Interface
UI	User Interface
PII	Personally Identifiable Information
CSV	Comma-Separated Values
PDF	Portable Document Format
ROC-AUC	Receiver Operating Characteristic – Area Under Curve

Terminology: In this report, “screening” means organizing and comparing application information for recruiter review. A score is a prioritization aid, not proof that a candidate is qualified or unqualified.

CHAPTER 1 — INTRODUCTION

1.1 Overview of the Project

Recruitment is an important organizational process. A recruiter compares candidate information with the responsibilities and requirements of a role. When applications arrive in different formats and contain different wording, manual comparison can be slow and inconsistent.

An AI-based resume screening system can assist by converting resume text into structured information and comparing it with a clearly written job description. It can highlight matching skills, missing required terms, and relevant experience statements, allowing a recruiter to focus on evidence rather than repeatedly searching documents.

1.2 Project Motivation

The project explores how natural-language processing and simple scoring methods can support an initial review. The emphasis is on explainability: the system should show why a match score was produced and allow the recruiter to correct extraction errors.

- Reduce repetitive searching and sorting.
- Apply a consistent set of job-related criteria.
- Show evidence for each identified skill or keyword.
- Keep the final decision with a qualified human reviewer.

CHAPTER 1 — PROBLEM STATEMENT AND OBJECTIVES

1.3 Problem Statement

Manual resume screening can be time-consuming when the applicant volume is high. Different resume layouts, abbreviations, spelling variations, and descriptions of similar skills make comparison difficult. Keyword-only screening may also miss candidates who describe equivalent experience using different words.

The project addresses the need for a basic, transparent tool that can parse resume text, compare it with a job description, and present a ranked review list without making an autonomous hiring decision.

1.4 Objectives

- Accept resume text and job descriptions as input.
- Normalize text and extract job-related skills and terms.
- Calculate an understandable match score using documented weights.
- Display matching evidence and possible missing requirements.
- Provide a recruiter review workflow with manual override.
- Identify fairness, privacy, and reliability risks before deployment.

1.5 Expected Outcome

The expected outcome is an academic prototype demonstrating resume parsing, job-description matching, score explanation, and a review interface. Its usefulness must be measured with suitable data before any real recruitment use.

CHAPTER 1 — SCOPE AND SIGNIFICANCE

1.6 Scope of the Project

The prototype focuses on text-based resumes and a single job description at a time. It may accept pasted text or extracted text from common document formats. It compares explicit job-related skills and phrases, and it presents a review list for a recruiter.

Included

- Text cleaning and basic keyword/skill extraction.
- A configurable list of required and preferred skills.
- A transparent score with component-level explanation.
- Manual review, correction, and export of results.

Outside the Current Scope

- Autonomous hiring, rejection, or interview decisions.
- Personality prediction or inference of protected traits.
- Guaranteeing candidate performance from resume wording.
- Production integration with applicant-tracking systems.

1.7 Significance

A well-designed screening aid can help recruiters organize information consistently and reduce repetitive work. Its value depends on good job criteria, accurate text extraction, and careful human review. A high score should never be treated as a guarantee of success, and a low score should not automatically exclude an applicant.

CHAPTER 1 — EXISTING APPROACHES AND LIMITATIONS

1.8 Existing System

In a traditional process, recruiters open resumes individually, search for qualifications, compare experience with a job description, and record decisions in spreadsheets or an applicant-tracking system. This approach can be effective when the applicant volume is small and criteria are clear.

Limitations

- Review time grows as the number of applications increases.
- Similar skills may be written in different ways.
- Different reviewers may apply criteria inconsistently.
- Manual records can contain duplicate entries or mistakes.
- A simple keyword filter can overlook relevant transferable skills.

1.9 Need for the Proposed System

The proposed system offers a repeatable first-pass comparison and highlights the text that influenced the result. It does not eliminate the need for recruiter judgment. The recruiter must confirm whether extracted information is accurate and consider relevant evidence that the model may have missed.

The design therefore includes explainability, manual override, privacy-conscious handling, and a clear warning that the score is only a support signal.

CHAPTER 2 — LITERATURE REVIEW

2.1 Literature Overview

Resume screening combines document processing, information extraction, text representation, similarity measurement, and decision support. Early automated approaches commonly used rules and keyword matching. These methods are easy to interpret but can be brittle when candidates use synonyms or different terminology.

Statistical text methods such as TF-IDF represent documents according to the importance of words in a collection. Cosine similarity can compare resume text with a job description. These approaches are useful baselines, although they may not fully capture context or the meaning of a sentence.

Modern language models and sentence embeddings can represent semantic relationships between phrases. For example, a resume may describe “building predictive models” while a job description asks for “machine learning experience.” Semantic methods can identify related wording, but their outputs still require validation and explanation.

2.2 Review Focus

- Rule-based systems: transparent and straightforward to maintain.
- TF-IDF and cosine similarity: efficient lexical similarity baseline.
- Skill dictionaries: useful for extracting explicit job-related evidence.
- Embedding models: potentially better at semantic matching.
- Human-in-the-loop systems: preserve accountability and context.

CHAPTER 2 — LITERATURE SURVEY

2.3 Thematic Survey

Information retrieval research provides the foundation for comparing documents against a query or reference text. In this project, the job description acts as the reference. A lexical baseline can identify shared terms and provide a reproducible score.

NLP techniques support normalization, tokenization, phrase extraction, and recognition of skill names. Domain dictionaries are practical for a student prototype because the skill vocabulary can be reviewed and changed by the project team. However, dictionaries require maintenance and can miss new technologies or equivalent phrases.

Machine-learning ranking systems can learn patterns from historical hiring decisions, but historical data may contain past discrimination or inconsistent preferences. Training on such data without analysis can reproduce existing inequities. For this reason, the initial prototype uses explicit job criteria and avoids treating past hiring outcomes as unquestioned ground truth.

2.4 Key Design Lessons

- Use the job description to define relevant criteria.
- Keep criteria visible and editable by authorized reviewers.
- Separate extracted evidence from the final score.
- Test on varied resume formats and writing styles.
- Audit errors and differences across relevant groups where lawful and appropriate.

This survey is a concise technical overview rather than a claim that a full systematic literature review has been completed.

CHAPTER 2 — RESEARCH GAP

2.5 Research Gap

A simple keyword filter is easy to build but can reward exact wording over equivalent experience. A complex model may improve semantic matching but can be harder to explain and audit. The practical gap for this mini project is to combine understandable criteria, text matching, score explanations, and human review in one small workflow.

2.6 Proposed Contribution

- A modular pipeline from text input to recruiter review.
- A baseline score based on job-related skill coverage and text similarity.
- A display of matched skills and criteria requiring review.
- A manual correction step for parsing and matching errors.
- A documented privacy and fairness checklist.

2.7 Research Questions

- Can the system extract relevant skills from typical text resumes?
- Does the score agree with a structured human review on a test sample?
- Which resume formats or wording patterns cause errors?
- Can users understand why a resume received its displayed score?

These questions should be evaluated with a representative, permissioned dataset. This report does not claim that those experiments have already been conducted.

CHAPTER 3 — SYSTEM DESIGN

3.1 Proposed System

The proposed system receives a job description and one or more resumes. It extracts text, cleans it, identifies job-related skills, calculates a score, and displays an explanation. The recruiter can inspect the source evidence and record a human decision separately from the automated score.

3.2 Main Modules

- Input module: accepts job requirements and resume documents/text.
- Text extraction module: reads text and handles extraction errors.
- Preprocessing module: normalizes case, spacing, and punctuation.
- Skill extraction module: matches explicit skills and approved aliases.
- Scoring module: combines skill coverage and text similarity.
- Dashboard module: sorts applications and shows score explanations.
- Audit module: records criteria version, reviewer changes, and errors.

3.3 Design Principle

The system should be deterministic where practical, explainable to the reviewer, and limited to information relevant to the job. A score must not be used as the only basis for a hiring decision.

CHAPTER 3 — SYSTEM ARCHITECTURE

3.4 Architecture

The system follows a modular pipeline. Each stage has a defined input and output so that extraction, scoring, and presentation can be tested separately.

RESUME FILES + JOB DESCRIPTION

↓

TEXT EXTRACTION AND VALIDATION

↓

CLEANING / NORMALIZATION

↓

SKILL EXTRACTION + TEXT SIMILARITY

↓

EXPLAINABLE MATCH SCORE

↓

RECRUITER DASHBOARD + HUMAN REVIEW

3.5 Data Flow

Input data is validated before processing. Extracted text is normalized without removing meaningful technical terms. The skill matcher checks the text against a configurable vocabulary. The scoring module produces component scores and supporting evidence. The dashboard displays results, while the reviewer confirms, corrects, or disregards the recommendation.

For privacy, raw resumes should be stored only when necessary, access should be restricted, and retention periods should be defined. A deployment should provide a way to delete candidate data.

CHAPTER 3 — FUNCTIONAL AND NON-FUNCTIONAL REQUIREMENTS

3.6 Functional Requirements

- FR1: The user can enter a job title and job description.
- FR2: The system can accept resume text or supported files.
- FR3: The system validates empty, unreadable, or oversized inputs.
- FR4: It extracts and lists recognized skills.
- FR5: It calculates a score using documented criteria.
- FR6: It displays matched evidence and potential gaps.
- FR7: The recruiter can review and correct extracted information.
- FR8: The system can export a review summary.

3.7 Non-Functional Requirements

- Usability: labels and explanations should be understandable.
- Performance: normal test files should be processed promptly.
- Security: restrict access and protect stored personal information.
- Reliability: handle missing text and unsupported formats safely.
- Maintainability: keep skill lists and scoring weights configurable.
- Fairness: avoid protected traits and audit potential disparate errors.
- Accessibility: support keyboard navigation and readable contrast.

3.8 Constraints

The quality of results depends on resume text extraction, the chosen skill vocabulary, the quality of the job description, and the validity of scoring weights. The system cannot reliably infer a candidate's full abilities from a document alone.

CHAPTER 3 — HARDWARE AND SOFTWARE REQUIREMENTS

3.9 Hardware Requirements

Component	Minimum / Suggested Specification
Processor	Intel Core i3 / equivalent or better
RAM	4 GB minimum; 8 GB recommended
Storage	At least 1 GB free for code and sample files
Display	Standard laptop or desktop display
Network	Internet only if optional online services are used

3.10 Software Requirements

Software	Purpose
Operating system	Windows, Linux, or macOS
Python 3	Main programming language
scikit-learn	TF-IDF and cosine similarity baseline
pandas	Optional tabular result handling
Flask / Streamlit	Optional web interface
PDF/DOCX parser	Optional resume text extraction
Web browser	Access to the dashboard

The basic text-matching prototype can run locally without a GPU. If external language models are introduced, compute, licensing, security, and data-transfer requirements must be reviewed.

CHAPTER 3 — DATA DESIGN AND SCORING

3.11 Input Data

The job description should identify required skills, preferred skills, minimum qualifications where relevant, and responsibilities. Resumes may contain contact details, education, work history, projects, certificates, and skills. Only fields necessary for the review should be processed.

3.12 Example Scoring Model

An illustrative score can combine required-skill coverage (50%), preferred-skill coverage (20%), and job-description text similarity (30%). The weights are examples for demonstration and must be reviewed with recruiters before use.

Final score = 0.50 × required-skill coverage + 0.20 × preferred-skill coverage + 0.30 × text similarity. Each component is normalized to a 0–100 scale.

3.13 Explanation

For each resume, the interface should show the recognized skills, the source phrase supporting each match, the score components, and any extraction warnings. Missing terms should be labelled as “not found in extracted text,” not as proof that the candidate lacks the skill.

3.14 Data Quality

- Check duplicates and unreadable files.
- Preserve technical tokens such as C++, C#, SQL, and .NET.
- Support common aliases such as “ML” and “machine learning.”
- Avoid using names, photographs, addresses, or other irrelevant traits in scoring.

CHAPTER 4 — METHODOLOGY

4.1 Development Methodology

The project follows an iterative process: requirement analysis, data preparation, prototype development, testing, evaluation, and refinement. Each module is first tested independently and then integrated into the full workflow.

Step 1: Requirement Analysis

Define the target role, identify essential and optional requirements, and confirm which criteria are genuinely relevant to the work. Criteria should be written before reviewing test resumes to reduce ad hoc changes.

Step 2: Sample Data Preparation

Prepare synthetic or permissioned sample resumes with varied formats and equivalent skill descriptions. Remove unnecessary personal identifiers. Keep a record of data sources and permitted uses.

Step 3: Text Processing

Extract text, normalize spacing and case, preserve technical symbols, and flag documents where text extraction is incomplete. The reviewer should be able to see the extracted text when needed.

Step 4: Matching

Identify known skills and compute a baseline similarity score. Compare results with structured human assessments and review errors before changing weights.

CHAPTER 4 — TEXT PREPROCESSING AND FEATURE EXTRACTION

4.2 Text Preprocessing

Preprocessing prepares resume text for comparison. Typical operations include converting ordinary text to a consistent case, replacing repeated whitespace, and removing irrelevant formatting artefacts. Aggressive cleaning should be avoided because punctuation may be meaningful in technology names.

Example

Input: “Experience with Python, SQL and machine-learning projects.”

Normalized text: “experience with python sql and machine-learning projects.”

4.3 Skill Extraction

A curated skill dictionary can map alternate phrases to a common label. For example, “natural language processing” and “NLP” may map to the same concept. The dictionary should be versioned and reviewed by people familiar with the role.

4.4 Text Similarity

TF-IDF converts text into weighted word vectors. Cosine similarity measures the angle between two vectors; a higher value indicates more overlap in the represented terms. It is a baseline rather than a complete understanding of candidate suitability.

4.5 Limitations of Feature Extraction

- A skill may be mentioned in a negative or hypothetical context.
- A resume may use a synonym absent from the dictionary.
- Text extraction may scramble columns or omit scanned text.
- A keyword match does not establish depth of practical experience.

CHAPTER 4 — MATCHING ALGORITHM

4.6 Algorithm

Input: job description J, required skill set R, preferred skill set P, and resume text set C.

- 1. Validate the job description and resume input.
- 2. Extract and normalize text from each resume.
- 3. Detect required and preferred skills using the approved dictionary.
- 4. Compute required-skill coverage and preferred-skill coverage.
- 5. Compute text similarity between the job description and resume.
- 6. Combine component scores using documented weights.
- 7. Sort results for review and show the score explanation.
- 8. Allow a human reviewer to inspect evidence and record a decision.

4.7 Pseudocode

```
FOR each resume in resume_collection:\n    text = extract_text(resume)\n    clean_text = normalize(text)\n    found_requir
```

The algorithm produces a review aid. It must not automatically reject an applicant based solely on a threshold.

CHAPTER 4 — IMPLEMENTATION PLAN

4.8 Implementation Modules

The project can be implemented in Python. A command-line prototype is sufficient to validate scoring before adding a user interface.

- resume_reader.py: reads or extracts resume text.
- preprocess.py: normalizes text and preserves technical terms.
- skill_matcher.py: matches skills and aliases.
- scorer.py: calculates score components and total score.
- app.py: presents results in a local web interface.
- tests/: contains unit tests and sample inputs.

4.9 User Interface

The dashboard should show a candidate reference number, match score, matching skills, text similarity, extraction warnings, and a review status. Access to full resume text should be restricted to authorized users.

4.10 Error Handling

- Show a useful message for an empty resume.
- Flag files from which no readable text was extracted.
- Avoid crashing when a skill list is empty.
- Log technical errors without exposing resume content in logs.

The report describes a proposed implementation structure; it does not claim that a production-ready recruitment application has been deployed.

CHAPTER 4 — TESTING STRATEGY

4.11 Testing

Testing checks whether the system behaves as expected, not whether it can make a perfect hiring decision. Test cases should include common resume formats, spelling variations, missing sections, duplicate files, and resumes that use equivalent skills with different wording.

Test Cases

Test ID	Input / Condition	Expected Behaviour
T01	Resume contains required skills	Skills are listed with evidence
T02	Resume omits a required term	Term is marked not found
T03	Empty resume text	Validation warning appears
T04	Unreadable document	Extraction error is reported
T05	Alias such as NLP	Approved alias is recognized
T06	Duplicate resume	Duplicate is flagged if enabled
T07	No preferred skills configured	Score calculation remains valid
T08	Candidate requests correction	Reviewer can update extracted data

4.12 Evaluation Measures

Measure extraction success, skill-match precision and recall, agreement with structured human review, processing time, and user comprehension of explanations. If a ranking model is evaluated, use an appropriate ranking metric and a held-out test set. Do not invent performance values where no experiment has been run.

CHAPTER 4 — FAIRNESS, PRIVACY AND SECURITY

4.13 Fairness Considerations

Historical recruitment data may reflect unequal access to education, biased hiring practices, or preferences unrelated to job performance. A model can reproduce these patterns even if protected attributes are not directly supplied. Proxy variables may also create unfair effects.

- Do not score names, photographs, gender, age, caste, religion, disability, or other protected traits.
- Review whether criteria are essential for the job and accessible to different career paths.
- Test error patterns across relevant groups where lawful and ethically justified.
- Provide human review, correction, and an appeal or reconsideration process.

4.14 Privacy and Security

- Collect only information necessary for the recruitment task.
- Restrict access through authentication and role-based permissions.
- Encrypt data in transit and at rest when stored.
- Define retention and deletion rules before collecting real resumes.
- Avoid sending identifiable resumes to third-party services without authorization.

4.15 Human Oversight

Recruiters remain responsible for reviewing evidence and making decisions. Candidates should not be rejected automatically because a parser failed or a term was not found. The system should be documented, monitored, and periodically re-evaluated.

CHAPTER 4 — ILLUSTRATIVE RESULTS

4.16 Demonstration Example

The following example demonstrates how a score might be displayed. It is fictional and is not the result of a measured experiment.

Job requirement	Resume A evidence	Illustrative status
Python	“Built scripts using Python”	Matched
SQL	“SQL queries and joins”	Matched
Machine learning	“Classification project”	Needs reviewer confirmation
Communication	Not explicit in extracted text	Not found; review manually

An illustrative dashboard could display: Required skills 75/100; preferred skills 60/100; text similarity 68/100. With example weights of 50%, 20%, and 30%, the combined score would be 69.4/100. These values are invented for explanation only.

4.17 Interpretation

The score tells the reviewer how the resume text matches the selected criteria. It does not establish competence, work quality, or future performance. A candidate may have a skill that is not listed in the resume, while another candidate may mention a keyword without substantial experience.

4.18 Result Reporting

A real evaluation should report dataset size and origin, inclusion criteria, extraction failure rate, precision/recall, scoring agreement, limitations, and error analysis. Any reported results must be traceable to actual experiments.

CHAPTER 4 — DISCUSSION AND LIMITATIONS

4.19 Discussion

A transparent baseline is suitable for a mini project because the score can be explained and modified. A recruiter can see which terms were found, inspect the source phrase, and decide whether the evidence satisfies the role requirement. This can improve consistency only when the criteria themselves are relevant and consistently applied.

4.20 Limitations

- Resume language is an incomplete representation of a person's skills.
- Keyword matching can miss synonyms, context, and transferable experience.
- Semantic models may introduce opaque or unexpected associations.
- Scanned documents and complex layouts can reduce extraction quality.
- Scores depend strongly on job-description quality and scoring weights.
- The prototype is not validated for high-stakes hiring decisions.

4.21 Risk Mitigation

Use synthetic data during development, disclose the system's purpose, keep a human reviewer in the loop, audit results, and provide a correction mechanism. Before deployment, consult applicable employment, privacy, accessibility, and AI governance requirements.

A responsible system should support a fair review process rather than replace professional judgment.

CHAPTER 5 — CONCLUSION

5.1 Conclusion

The AI-Based Job Resume Screening System mini project presents a structured approach to comparing resume text with job-related requirements. Its main components are text extraction, preprocessing, skill matching, similarity scoring, explanation, and human review.

The proposed architecture can reduce repetitive searching and help recruiters organize applications. The use of configurable criteria and visible evidence makes the baseline easier to inspect than a fully opaque ranking model. However, the quality of output depends on the quality of the input text, the job description, and the matching rules.

The project also highlights that recruitment is a high-impact context. A score should not be the sole reason for rejection, and sensitive personal characteristics should not be used as screening features. Privacy, fairness, security, accessibility, and accountability must be addressed from the beginning.

5.2 Objectives Achieved in the Design

- Defined a modular resume-to-job-description workflow.
- Specified transparent skill coverage and similarity scoring.
- Outlined a recruiter dashboard and manual review process.
- Identified functional tests, evaluation measures, and key risks.

The report describes a design and illustrative prototype approach. A working implementation and empirical results must be demonstrated separately before claiming operational effectiveness.

CHAPTER 5 — FUTURE WORK

5.3 Future Enhancements

- Add reliable PDF and DOCX text extraction with OCR for scanned documents.
- Use sentence embeddings to compare equivalent phrases semantically.
- Add configurable role templates and versioned skill dictionaries.
- Provide a reviewer feedback mechanism to correct extraction errors.
- Evaluate ranking quality using a permissioned, representative dataset.
- Run fairness audits and monitor changes after each model update.
- Add access control, encryption, retention schedules, and deletion tools.
- Improve accessibility and support multiple languages where appropriate.

5.4 Deployment Considerations

Before deployment, the organization should conduct a documented impact assessment, involve recruiters and affected stakeholders, define accountability, and ensure candidates can seek human reconsideration. Model changes should be tested and approved before release.

5.5 Final Note

AI should help people make better-informed decisions, not make consequential recruitment decisions without meaningful oversight. The strongest future version of this project will combine technical performance with explainability, fairness, privacy, and clear accountability.

APPENDIX A — SAMPLE PYTHON CODE

A.1 Illustrative Skill-Matching Baseline

The short example below demonstrates a simple transparent skill-coverage score. It is educational sample code, not a complete recruitment system. Real applications need document extraction, stronger validation, access control, testing, and human review.

```
required_skills = {"python", "sql", "machine learning"}
preferred_skills = {"flask", "git", "data visualization"}

def find_skills(text, skill_set):
    text = text.lower()
    return {skill for skill in skill_set if skill in text}

def coverage(found, expected):
    if not expected:
        return 100.0
    return 100.0 * len(found) / len(expected)

resume_text = """
Built Python scripts and SQL queries for a machine learning project.
Used Git for version control.
"""

required_found = find_skills(resume_text, required_skills)
preferred_found = find_skills(resume_text, preferred_skills)

required_score = coverage(required_found, required_skills)
preferred_score = coverage(preferred_found, preferred_skills)

print("Required skills:", sorted(required_found))
print("Preferred skills:", sorted(preferred_found))
print("Required coverage:", round(required_score, 1))
print("Preferred coverage:", round(preferred_score, 1))
print("Human review is required before any decision.")
```

Note: This baseline uses literal substring matching and may miss synonyms or produce context errors. A production version should use tested phrase matching and display evidence for each match.

REFERENCES

Selected References and Further Reading

1. Manning, C. D., Raghavan, P., and Schütze, H. Introduction to Information Retrieval. Cambridge University Press, 2008.

2. Salton, G. and Buckley, C. “Term-weighting approaches in automatic text retrieval.” Information Processing & Management, 1988.

3. Pedregosa, F. et al. “Scikit-learn: Machine Learning in Python.” Journal of Machine Learning Research, 2011.

4. Bird, S., Klein, E., and Loper, E. Natural Language Processing with Python. O’Reilly Media, 2009.

5. Barocas, S., Hardt, M., and Narayanan, A. Fairness and Machine Learning. Online book, 2019.

6. National Institute of Standards and Technology. AI Risk Management Framework (AI RMF 1.0), 2023.

7. European Union. Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act), 2024.

8. Python Software Foundation. Python documentation.

9. scikit-learn developers. scikit-learn user guide: feature extraction and text similarity.

Project Report Note

This report follows the academic mini-project structure of the uploaded example: cover page, certificate, acknowledgement, abstract, contents, introduction, literature review, system design, implementation, conclusion, future work, code, and references. Replace bracketed placeholders with your approved student and guide details. Verify references and institutional formatting before submission.